Per-Function Calling Convention Optimization for Irregular Register Architectures

Target venue: CC 2027 or LCTES 2027 **Draft status:** v0.1 — structure + content, needs polish

Abstract

Compilers for register-constrained architectures with irregular register files — where registers have distinct semantic roles (accumulator, counter, pointer) — typically use a single fixed calling convention for all functions. We formalize *Per-Function Calling Convention Optimization* (PFCCO): given a program's call graph and a cost model over register classes, choose a calling convention for each internal function to minimize total transfer cost. We show that PFCCO is solvable optimally in O(n) time for acyclic call graphs with bounded parameter counts, by greedy dynamic programming in reverse topological order. Our implementation in the Nanz compiler targeting the Zilog Z80 produces code that is 1.3–17× smaller and up to 17× faster than SDCC 4.2.0 (which uses a state-of-the-art empirically-optimized global convention), with emergent optimizations including zero-byte function aliases.

1. Introduction

Register allocation is among the most studied problems in compiler construction. For *intraprocedural* allocation — assigning physical registers to virtual registers within a single function — optimal algorithms exist for structured programs on irregular architectures (Krause 2013). For the *interprocedural* dimension — how values are passed between functions — the standard approach is a fixed calling convention: every function receives its first argument in the same register, returns its result in the same register, and so on.

On architectures with large, regular register files (ARM, x86-64, RISC-V), the calling convention is primarily about partitioning registers into caller-saved and callee-saved sets. The choice of *which* register holds a parameter matters little because all general-purpose registers have identical capabilities.

On *irregular* architectures — microcontrollers like the Zilog Z80, Intel 8080, Padauk, or SM83 — registers have semantic roles enforced by the instruction set:

- The **accumulator** (A) is the only register that can be an ALU source/destination
- The **counter** (B) is the only register usable with DJNZ (decrement-and-jump)
- The **pointer pair** (HL) is the only pair usable for memory addressing with LD r,(HL)
- The **index pair** (DE) is needed as the second operand of ADD HL,DE / SBC HL,DE

Choosing the wrong register for a parameter forces the compiler to insert *adapter moves* — register-to-register transfers at call sites or inside the function body. On a Z80 with only 7 main registers, each unnecessary LD costs 4 T-states and 1 byte. These costs compound across the call graph.

Krause (2022) addressed this by empirically searching for the best *single* calling convention per architecture, testing thousands of configurations against standard benchmarks. This improved SDCC's Z80 backend significantly — enough to justify breaking ABI compatibility. But a global convention is inherently a compromise: the best choice for a leaf function doing arithmetic differs from the best choice for a function that forwards arguments to a callee.

We observe that internal functions (those not exported across compilation units) need not share a convention at all. Each function can have its own *contract* — a mapping from parameter/return slots to register classes — chosen to minimize the total cost of adapter moves plus call-site transfers across the entire program.

This paper makes the following contributions:

1. **Formalization** of Per-Function Calling Convention Optimization (PFCCO) as a cost-minimization problem over register class assignments on a call graph (§3).
2. **An O(n) algorithm** (for bounded parameter counts on acyclic call graphs) based on greedy DP in reverse topological order, with a proof of optimality for DAGs (§4).
3. **A production implementation** in the Nanz compiler targeting Z80, demonstrating 1.3–17× code size reduction and up to 17× speedup over SDCC 4.2.0 on representative examples (§5–6).
4. **Identification of emergent optimizations** — notably zero-byte function aliases (EQU collapse) that arise naturally when the optimizer aligns caller and callee ABIs (§6.4).

2. Background

2.1 The Z80 Register File

The Zilog Z80 (1976) has 14 8-bit registers organized as:

Register	Role	Key instructions
A	Accumulator	ADD, SUB, AND, OR, XOR, CP, NEG
B	Counter	DJNZ (decrement B, jump if nonzero)
C, D, E	General	LD r,r' only
H, L	Pointer pair	LD A,(HL), LD (HL),A, INC HL
D, E	Index pair	ADD HL,DE, SBC HL,DE, EX DE,HL
B, C	Counter pair	ADD HL,BC, PUSH BC

Additionally, a *shadow register bank* (A', B', C', D', E', H', L') is accessible via the EXX instruction, which atomically swaps all three main pairs with their shadows. We discuss shadow registers in §8 (Future Work).

The register file is *irregular*: no two registers are interchangeable. Moving a value from C to A costs 4 T-states (LD A,C); moving from HL to DE costs 4T (EX DE,HL) or 8T (LD D,H; LD E,L). These asymmetries mean that the *choice* of register for a parameter directly affects instruction selection and code quality.

2.2 Fixed vs. Per-Function Calling Conventions

A *calling convention* (or *ABI*) specifies, for each function, which registers hold parameters and return values.

Fixed (global) convention. One convention for all functions. SDCC 4.2.0 uses Krause (2022)'s empirically-optimized convention:

Slot	8-bit	16-bit
Param 1	A	HL
Param 2	L	DE
Param 3+	stack	stack
Return	A	DE

Per-function convention. Each internal function may use a different register assignment. External functions (library, ROM, interrupt handlers) retain a fixed convention.

2.3 The Nanz Compiler

Nanz is a systems language targeting Z80-class microprocessors. Its compilation pipeline is:

```
Nanz source → Parser → HIR → MIR2 → Z80 assembly
                              ↑
                        PFCCO runs here
                        (OptimizeContracts)
```

The HIR (High-level IR) lowers to MIR2 (Machine-level IR), which represents operations in terms of virtual registers with *register class* annotations. After PFCCO assigns per-function contracts, the PBQP register allocator maps virtuals to physicals within each function, respecting the contract.

2.4 Register Classes

We define the following register classes for Z80:

Class	Physical registers	Type affinity
ClassAcc	A	u8 (ALU operand)
ClassCounter	B	u8 (loop count)
ClassGeneral	C, D, E	u8 (general)
ClassPointer	HL	u16, ptr (memory access)
ClassIndex	DE	u16 (arithmetic)
ClassPair	BC	u16 (general pair)

Each class has a *plausible type set*: ClassAcc is plausible for u8 parameters, ClassPointer for u16/ptr, etc.

---

3. Problem Formulation

3.1 Definitions

Definition 1 (Contract). A *contract* for function f with k parameter slots and r return slots is a vector $C(f) = (c_1, \ldots, c_k, c_{r1}, \ldots, c_{rr})$ where each c_i is a register class compatible with the slot's type.

Definition 2 (Adapter cost). The *adapter cost* of contract C(f) is the sum of move costs inside f's body when the assigned class does not match the class naturally required by the body's instructions:

$$\text{adapterCost}(f, C) = \Sigma_i \, \text{moveCost}(c_i, \text{naturalClass}(f, i)) + \Sigma_j \, \text{moveCost}(\text{naturalRetClass}(f, j), c_{rj}) \times |\text{retSites}(f)|$$

Definition 3 (Transfer cost). The *transfer cost* at a call site e = (caller, callee) is:

$$\text{transferCost}(e) = \Sigma_k \, \text{moveCost}(\text{argClass}(\text{caller}, k), C(\text{callee}).\text{param}_k)$$

where argClass is the register class of the k-th argument at the call site.

Definition 4 (PFCCO). Given call graph G = (F, E), register file R, and cost function moveCost, find an assignment $C : F \to \text{Contracts}$ minimizing:

$$\text{totalCost}(C) = \Sigma_{f \in F} \, \text{adapterCost}(f, C(f)) + \Sigma_{e \in E} \, \text{transferCost}(e, C) \times w(e)$$

subject to: no two parameter slots of the same function map to the same uniquely-forced physical register.

3.2 The naturalClass Function

The bridge between intraprocedural instruction selection and interprocedural ABI optimization is

`naturalClass(f, i)` — the register class that function f's body "wants" for its i-th parameter:

```
naturalClass(f, i):
  for each instruction using param i (in block
order):
    if ALU operation (ADD/SUB/AND/OR/XOR/CP): return
ClassAcc
    if memory access (Load/Store/PtrAdd):     return
ClassPointer
    if loop counter (DJNZ):                   return
ClassCounter
    if forwarded as arg k to callee g:        return
C(g).paramₖ  // TRANSITIVITY
  return ClassGeneral  // default
```

The transitivity case is the crucial insight: if parameter i of function f is forwarded directly as argument k to callee g, the natural class is whatever g's contract expects. This creates chains of aligned ABIs through the call graph without explicit coordination.

---

4. Algorithm

4.1 Greedy DP on Reverse Topological Order

```
PFCCO-Optimize(Module M):
  G ← BuildCallGraph(M)
  order ← TopoSort(G, leaves_first=true)    //
Kahn's on reverse graph
  CS ← {}                                   //
contract set

  for f in order:
    if f.isExtern or f.isEmpty:
      CS[f] ← currentClasses(f)             // fixed
ABI
      continue

    bestChoice ← currentClasses(f)          // Layer
1 default
    bestCost ← Cost(f, bestChoice, G, CS)

    for candidate in
CartesianProduct(plausibleClasses(f)):
      if HasParamConflict(candidate):
        continue
      cost ← Cost(f, candidate, G, CS)
      if cost < bestCost:
        bestCost ← cost
        bestChoice ← candidate

    CS[f] ← bestChoice

  return CS
```

4.2 Three-Layer ABI System

The algorithm operates within a three-layer system:

- **Layer 1: Positional defaults.** A heuristic assigns classes based on parameter position and type (e.g., first u8 → ClassAcc, first u16 → ClassPointer). This provides the initial contract.
- **Layer 2: User annotations.** The programmer may override classes with annotations (e.g., `@z80_a`, `@z80_hl`). These are hard constraints respected by the optimizer.
- **Layer 3: Contract optimizer (PFCCO).** The algorithm above, which searches for better class assignments over the call graph.

4.3 Complexity

For a function with P parameter slots and R return slots: - Candidates per slot: at most 3 (u8: {Acc, Counter, General}) - Total candidates: ≤ 3^(P+R), reduced by conflict filtering - Cost evaluation: O(|body| + |call_sites|)

For bounded P, R (typical: P ≤ 4, R ≤ 2): - Per function: O(3^6 × |body|) = O(|body|) - Total: **O(n)** where n = total program size (instructions)

4.4 Optimality for Acyclic Call Graphs

Theorem. For acyclic call graphs, PFCCO-Optimize produces an optimal solution.

Proof. Topological order (leaves first) ensures every callee g is decided before any caller f that calls g. When processing f, we evaluate all candidate assignments against exact costs: adapter costs within f are computed directly, and transfer costs to callees use their final (optimal) contracts. Since f's choice affects only (a) f's internal adapter cost and (b) transfer costs on edges from f to its callees — not costs within callees or between other function pairs — the locally optimal choice for f is globally optimal. This is a standard bottom-up DP argument on DAGs. ■

Recursive functions. Cycle members are appended to the topological order with their intra-cycle callees' contracts unknown. The algorithm uses default contracts for undecided cycle members, yielding a heuristic result. Tarjan's SCC decomposition with fixed-point iteration within each SCC would restore optimality, at the cost of O(iterations × |SCC|).

---

5. Implementation

5.1 Nanz Compiler Integration

PFCCO is implemented in 455 lines of Go in the `mir2` package of the Nanz compiler. Key components:

Component	Lines	Function
`OptimizeContracts`	56–87	Main algorithm loop
`candidateChoices`	132–178	Cartesian product + conflict filter
`plausibleClasses`	226–240	Per-type class enumeration
`inferNaturalClass`	344–386	Body analysis + transitivity
`edgeCost`	289–331	Call-site transfer cost
`classMoveCost`	430–438	Physical move cost lookup
`BuildCallGraph`	31–68	Call graph construction
`topoSort`	103–156	Kahn's algorithm

5.2 Cost Table

Move costs between classes (Z80 T-states):

From \ To	Acc	Counter	General	Pointer	Index
Acc	0	4	4	—	—
Counter	4	0	4	—	—
General	4	4	0†	—	—
Pointer	—	—	—	0	4‡
Index	—	—	—	4‡	0
Pair	—	—	—	11§	11§

† 0 if same register, 4 if different (C→D) ‡ EX DE,HL (4T) or LD D,H; LD E,L (8T) § PUSH BC; POP HL (21T) or per-byte copy

5.3 Pipeline Position

PFCCO runs after HIR→MIR2 lowering and before PBQP register allocation:

```
HIR → MIR2 lowering (Layer 1 defaults) → PFCCO
(Layer 3) → PBQP RA → Z80 codegen
```

This ordering is essential: PFCCO sets the *contracts* (interface classes), then the allocator maps virtuals to physicals *within* each function, respecting the contract constraints.

---

6. Evaluation

We compare Nanz (with PFCCO) against SDCC 4.2.0, which uses Krause (2022)'s empirically-optimized global calling convention — the current state of the art for Z80 ABI selection.

6.1 abs_diff (u8): Leaf Function

```
fun abs_diff(a: u8, b: u8) -> u8 {
    if a < b { return b - a }
    return a - b
}
```

	Nanz (PFCCO)	SDCC 4.2.0
Contract	A=a, C=b	A=a, L=b
Bytes	**4**	10
T-states (worst)	**27T**	34T

PFCCO chose ClassGeneral(C) for the second parameter. `SUB C` is a single-byte instruction that sets carry for the conditional return. SDCC's global convention places the second u8 in L, which cannot be a direct SUB operand — forcing a save-to-C and redundant recomputation.

6.2 GCD: Loop-Carried Register Persistence

```
fun gcd(a: u8, b: u8) -> u8 {
    while a != b {
        if a > b { a = a - b }
        else     { b = b - a }
    }
    return a
}
```

	Nanz	SDCC
Contract	A=a, C=b	C=a, L=b
Bytes	14	19
Reload per iteration	**0**	1 (LD A,C)

SDCC's global ABI forces `a` into C at function entry (A is the first-arg register but L is the second, creating a conflict with ALU usage). Every loop iteration requires `LD A,C` to reload the accumulator. PFCCO keeps `a` in A throughout.

6.3 forEach: Iterator Fusion with ClassCounter

```
fun sum_chain(buf: ^u8, n: u8) -> u8 {
    var s: u8 = 0
    buf.forEach(|x: u8| { s = (s + x) }, n)
    return s
}
```

	Nanz	SDCC
Bytes	**12**	29
T per element	**~38T**	~88T
Loop control	DJNZ (13T)	INC B; SUB 4(IX); JR (35T)
Array access	Sequential (INC HL)	Indexed (HL=buf+i each iter)

PFCCO assigned `n` to ClassCounter(B), enabling Z80's DJNZ instruction. Combined with iterator fusion (sequential pointer traversal instead of indexed access), the per-element cost is 2.3× lower.

6.4 swap / minmax: Multi-Return and EQU Collapse

```
fun minmax(a: u16, b: u16) -> (u16, u16) {
    if a <= b { return (a, b) }
    return (b, a)
}

fun min_of(a: u16, b: u16) -> u16 {
    let (lo, _) = minmax(a, b)
    return lo
}
```

Function	Nanz	SDCC
swap	`EX DE,HL; RET` — **2 bytes, 14T**	Frame + stores — 26 bytes, 236T
min_of	`JP minmax` — **3 bytes** (0 with EQU)	~40 bytes, ~200T

EQU collapse. Because PFCCO aligned `min_of`'s contract with `minmax`'s (both: param1=HL, param2=DE, return=HL), the tail call `min_of → minmax` requires no argument setup and no result extraction. The assembler can replace `min_of: JP minmax` with `min_of EQU minmax` — a **zero-byte function**. This is an emergent property: nobody programmed "detect alias opportunities." It falls out of cost minimization.

C cannot express multi-return, forcing SDCC to use pointer out-parameters with stack frames. The 17× speedup on `swap` is partly a language-level advantage (multi-return syntax) and partly an ABI-level advantage (per-function register assignment).

6.5 Fibonacci: Recursive ABI Alignment

	Nanz	SDCC
Return register	HL	DE
EX DE,HL after each recursive CALL	**0**	1 (4T)
Stack saves per recursive call	0	2 (PUSH DE + PUSH HL = 22T)

Nanz returns u16 in HL (matching `ADD HL,DE`'s expectation). SDCC's global convention returns in DE, requiring `EX DE,HL` after every recursive call.

6.6 Summary

Example	Nanz bytes	SDCC bytes	Size ratio	T-state ratio
abs_diff (u8)	4	10	**2.5×**	1.3×
abs_diff (u16)	10	16	**1.6×**	1.2×
gcd	14	19	**1.4×**	~1.5× per iter
swap	2	26	**13×**	**17×**
min_of	3 (0)	~40	**13×+**	**20×+**
forEach	12	29	**2.4×**	**2.3×**
fib	~26	~28	1.1×	~1.2× per call

Per-function ABI's advantage scales with structural complexity: more functions, deeper call graphs, multi-return. For isolated leaf functions, SDCC's global ABI is near-optimal.

7. Related Work

Intraprocedural RA on irregular files. Krause (2013) presents optimal register allocation via tree decomposition for structured programs, implemented in SDCC. PFCCO is complementary: it sets the calling convention that Krause's allocator then respects.

Global ABI search. Krause (2022) empirically evaluates thousands of calling conventions per architecture, selecting the best single convention. PFCCO strictly generalizes this: any global ABI is a special case where all functions receive the same assignment.

Link-time per-function CC. De Bus et al. (2004) and Caldwell & Chiba (2017) optimize calling conventions per-function at link time for ARM. Both target regular RISC register files where the problem is primarily caller/callee-save partitioning, not register *class* assignment. Neither provides formal optimality results.

Interprocedural RA for RISC. Wall (1986) proposes link-time register allocation for MIPS (32 GPRs). LLVM IPRA (2016) propagates clobbered-register sets to reduce caller-save overhead. Both target large, regular files where all registers are interchangeable — the *which register* question is irrelevant.

PBQP. Scholz & Eckstein (2002) formulate register allocation as PBQP. PFCCO's cost function has natural PBQP structure (unary adapter costs + edge transfer costs). For the small candidate spaces of Z80 (≤729 per function), exhaustive enumeration suffices; PBQP solvers would be useful for architectures with larger plausible class sets.

MSVC /GL. Microsoft's whole-program optimization promotes stack parameters to registers for internal functions. This is a special case of per-function CC optimization for x86's stack-default ABI. No formal analysis is published.

8. Future Work

Shadow register bank integration. The Z80's shadow register bank (A', B', C', D', E', H', L') provides 7 additional storage locations accessible via the EXX instruction. Our implementation uses shadow registers for 32-bit values (ClassDWord: HL+H'L') and 8-bit overflow (ClassShadow), but these assignments are currently *hardcoded by type*, not optimized by PFCCO. Extending the optimizer to include shadow classes in its candidate enumeration requires modeling the EXX atomicity constraint: switching one shadow pair forces all three pairs to switch simultaneously, at 4 T-states per direction. This bank-level switching constraint makes shadow register optimization a distinct research problem.

Recursive function optimization. The current algorithm uses default contracts for undecided cycle members. SCC decomposition with fixed-point iteration would restore optimality for mutually-recursive clusters.

Profile-guided weighting. The current implementation uses unit edge weights. Dynamic profiling would weight hot call edges, potentially shifting ABI choices for performance-critical paths.

Benchmark suite evaluation. Our evaluation uses representative examples rather than standard benchmark suites (Whetstone, Dhrystone, Coremark). A comprehensive evaluation against the benchmarks used by Krause (2022) would provide a direct comparison of per-function vs. global ABI optimization on identical workloads.

9. Conclusion

We have formalized Per-Function Calling Convention Optimization as a cost-minimization problem on call graphs with register class assignments, proved its optimal solvability for acyclic call graphs in linear time, and demonstrated its effectiveness in a production compiler for the Z80 — an architecture that has been a challenging target for compilers for nearly 50 years.

The key insight is that on irregular register files, the calling convention is not merely an interface specification but an optimization variable with direct impact on instruction selection. By treating it as such, the compiler discovers optimizations — including zero-byte function aliases — that no fixed convention can achieve.

References

1. Krause, P. (2013). Optimal Register Allocation in Polynomial Time. CC 2013.
2. Krause, P. (2022). Efficient Calling Conventions for Irregular Architectures. arXiv:2112.01397.
3. De Bus, B., et al. (2004). Link-time optimization of ARM executables.
4. Caldwell, A. & Chiba, S. (2017). Reducing calling convention overhead in object-oriented programs.
5. Wall, D.W. (1986). Global Register Allocation at Link Time. SIGPLAN Notices.
6. Scholz, B. & Eckstein, E. (2002). Register Allocation for Irregular Architectures. LCPC/SCOPES.
7. US Patent 5,428,793 (1995). Interprocedural register allocation.